

Integración de Extracción con Azure AI Content Understanding

Índice

- 1. Descripción general
- 2. Arquitectura del flujo
- 3. Componentes principales
- 4. Configuración y seed
- 5. Añadir soporte para una nueva tipología
- 6. Configuración en local (desarrollo)
- 7. Configuración en Azure (producción)
- 8. Mapeo de campos
- 9. Proveedor mock (sin Azure)
- 10. Preguntas frecuentes

1. Descripción general

El pipeline de procesamiento de documentos incluye un paso de **extracción de datos estructurados** a partir del contenido del documento. Este paso está soportado por **Azure AI Content Understanding**, un servicio de Azure AI Foundry que analiza documentos en formato PDF u otros formatos y devuelve los campos definidos en un analizador previamente entrenado.

La integración es **totalmente guiada por configuración**: no hay código específico por tipología. Añadir una nueva tipología con extracción Azure solo requiere:

- Definir un analizador en Azure AI Foundry.
- Registrar ese analizador en la tabla `ModeloConfigs` mediante Admin API/DocumentIA.Admin.
- Habilitar la extracción en `Tipologias.ConfiguracionJson` para la tipología publicada.

Los ficheros `models.json` y `*.validation.json` del repositorio son seed o referencia historica; pueden estar desactualizados y no son la fuente de verdad operativa.

2. Arquitectura del flujo



Datos que fluyen

Paso	Entrada	Salida
ExtraerActivity	ExtraccionInput (tipología + documento en base64 + datos normalizados previos)	ExtraccionResultado
AzureContentUnderstandingProvider	ExtraccionInput	ExtraccionResultado con DatosExtraidos
ContentUnderstandingResultMapper	JSON de respuesta de Azure + TipologiaValidationConfig	Dictionary<string, object>

3. Componentes principales

ConfigurableExtraerDataProvider

Fichero: DocumentIA.Functions/Services/ConfigurableExtraerDataProvider.cs

Actúa como router. Lee el campo `Extraction.Provider` de la configuración publicada de la tipología en BD y delega en el proveedor correspondiente. Si el campo está vacío, usa el valor de `Extraction:DefaultProvider` de la configuración de la Function App.

Proveedores soportados:

Valor en config	Proveedor	Descripción
azure-content-understanding	AzureContentUnderstandingProvider	Llamada a Azure AI Content Understanding. Si la completitud es baja puede activar fallback GPT.
azure-document-intelligence / azure-di	AzureDocumentIntelligenceExtraerDataProvider	Extracción con Azure Document Intelligence (layout/modelo).
azure-openai / openai / gpt	GptDirectExtraerDataProvider	Extracción directa con GPT (sin CU previo). Requiere <code>Extraction:GptFallback:Endpoint/DeploymentName/ApiKey</code> .
mock	MockExtraerDataProvider	Datos fijos para testing.

AzureContentUnderstandingProvider

Fichero: DocumentIA.Functions/Services/AzureContentUnderstandingProvider.cs

Implementa la llamada real al servicio Azure AI Content Understanding:

- 1. Carga la configuración de la tipología para obtener `ModelKey`.
- 2. Resuelve el modelo en `ExtractionModelRegistryLoader` para obtener `AnalyzerId`, `ContentType`, etc.
- 3. Decodifica el campo `Documento.Content.Base64` del contrato de entrada.

Nota: cuando la petición llega por `documento.objectIdGDC`, el orquestador recupera previamente el binario desde GDC y lo hidrata en `Documento.Content.Base64` antes de entrar en las actividades de normalización/extracción.

- 4. Llama a `ContentUnderstandingClient.AnalyzeBinaryAsync` con `WaitUntil.Completed` (operación de larga duración, espera hasta que Azure finaliza el análisis).
- 5. Parsea el JSON de respuesta y lo pasa al mapper.

ContentUnderstandingResultMapper

Fichero: DocumentIA.Functions/Services/ContentUnderstandingResultMapper.cs

Transforma la respuesta de Azure en el diccionario `DatosExtraidos`. Para cada campo definido en la tipología:

- Busca el campo en `result.contents[0].fields` del JSON de respuesta.

- Si existe una `fieldMapping` explícita, sigue el `SourcePath` indicado (notación de punto, p. ej. `Owner.Name`).
- Si `AutoMapUnmappedFields` es `true` , intenta localizar el campo por el mismo nombre que tiene en la configuración.
- Convierte el valor al tipo CLR apropiado según el tipo de campo de Azure CU (`valueString` , `valueDate` , `valueNumber` , `valueBoolean` , `valueArray` , `valueObject` , `content` , etc.).

ExtractionModelRegistryLoader

Fichero: `DocumentIA.Core/Configuration/ExtractionModelRegistryLoader.cs`

Carga y cachea en memoria los modelos publicados en la tabla `ModeloConfigs` (tipo `Extraccion`). Proporciona el método `GetModel(string key)` que lanza `KeyNotFoundException` si la clave no existe. El fichero `config/extraction/models.json` queda como seed/bootstrap o referencia historica.

4. Configuración y seed

4.1 Registro de modelos de extraccion

Fuente de verdad runtime: tabla `ModeloConfigs` (`TipoModelo.Extraccion`), gestionada por `DocumentIA.Admin` o `Admin API`.

Seed/referencia historica: `DocumentIA.Functions/config/extraction/models.json` .

Cada entrada describe un analizador de Azure AI Content Understanding.

```
{
  "models": [
    {
      "key": "nota.simple.1_4.azure-cu",
      "provider": "azure-content-understanding",
      "analyzerId": "nota-simple-1-4",
      "contentType": "application/pdf",
      "processingLocation": "global",
      "inputRange": ""
    }
  ]
}
```

Campo	Obligatorio	Descripción
<code>key</code>	Sí	Identificador único. Referenciado desde los ficheros de validación de tipología en <code>extraction.modelKey</code> .
<code>provider</code>	Sí	Debe ser <code>azure-content-understanding</code> .
<code>analyzerId</code>	Sí	ID del analizador tal como está creado en Azure AI Foundry.
<code>contentType</code>	Sí	MIME type del documento (<code>application/pdf</code> , <code>image/jpeg</code> , etc.). Si está vacío se intenta detectar automáticamente.
<code>processingLocation</code>	No	Región de procesamiento. Si está vacío, usa <code>DefaultProcessingLocation</code> de la configuración de la Function App (por defecto <code>global</code>).
<code>inputRange</code>	No	Rango de páginas a analizar (p. ej. <code>"pages": "1-3"</code>). Vacío = documento completo.

4.2 Fichero de validación de tipología

Ruta: `DocumentIA.Functions/config/tipologias/<tipologia>.validation.json`

Ejemplo del bloque `extraction` en `nota.simple.1_4.validation.json` :

```
{
  "tipologiaId": "notasimple",
  "version": "1.4",
  "extraction": {
    "enabled": true,
    "provider": "azure-content-understanding",
    "modelKey": "nota.simple.1_4.azure-cu",
    "autoMapUnmappedFields": true,
    "fieldMappings": []
  },
  "fields": [ ... ]
}
```

Campo	Obligatorio	Descripción
<code>enabled</code>	Sí	<code>true</code> para activar la llamada a Azure. <code>false</code> deja el paso sin extracción.
<code>provider</code>	Sí	<code>azure-content-understanding</code> o <code>mock</code> .
<code>modelKey</code>	Sí	Clave del modelo en <code>models.json</code> .
<code>autoMapUnmappedFields</code>	No	Si <code>true</code> (por defecto), intenta casar automáticamente campos de Azure con los campos declarados en <code>fields[]</code> por nombre coincidente.
<code>fieldMappings</code>	No	Mapeos explícitos cuando el nombre del campo en Azure difiere del nombre en la tipología (ver sección 8).

4.3 appsettings.json

Ruta: DocumentIA.Functions/appsettings.json

Valores por defecto (sin datos sensibles). Se usan fuera del runtime de Azure Functions (p. ej., pruebas de integración).

```
{
  "Extraction": {
    "DefaultProvider": "mock",
    "AzureContentUnderstanding": {
      "Endpoint": "",
      "ApiKey": "",
      "AuthMode": "ApiKey",
      "DefaultProcessingLocation": "global"
    }
  }
}
```

4.4 local.settings.json

Ruta: DocumentIA.Functions/local.settings.json

Configuración local para desarrollo. Este fichero **no se sube al repositorio** (está en `.gitignore`).

Las claves relevantes dentro de `Values` son:

```
{
  "Values": {
    "Extraction:DefaultProvider": "mock",
    "Extraction:AzureContentUnderstanding:Endpoint": "https://<tu-recurso>.cognitiveservices.azure.com/",
    "Extraction:AzureContentUnderstanding:ApiKey": "<tu-api-key>",
    "Extraction:AzureContentUnderstanding:AuthMode": "ApiKey",
    "Extraction:AzureContentUnderstanding:DefaultProcessingLocation": "global"
  }
}
```

5. Añadir soporte para una nueva tipología

Supongamos que queremos añadir extracción Azure para una tipología `tasacion.1_0`.

Paso 1: Crear el analizador en Azure AI Foundry

En el portal de Azure AI Foundry, crear un analizador con el ID que quieras usar (p. ej. `tasacion-1-0`) y entrenarlo con los documentos representativos. Anotar el `analyzerId` resultante.

Paso 2: Registrar el modelo en BD

Crear o actualizar una entrada en `ModeloConfigs` mediante `DocumentIA.Admin` o `Admin API`. Puede usarse este JSON como plantilla o seed de un entorno nuevo:

```
{
  "key": "tasacion.1_0.azure-cu",
  "provider": "azure-content-understanding",
  "analyzerId": "tasacion-1-0",
  "contentType": "application/pdf",
  "processingLocation": "global",
  "inputRange": ""
}
```

Paso 3: Habilitar la extracción en la configuración de la tipología

En `Tipologias.ConfiguracionJson`, añadir el bloque `extraction`. El fichero `config/tipologias/tasacion.1_0.validation.json` solo sirve como plantilla/seed:

```
{
  "tipologiaId": "tasacion",
  "version": "1.0",
  "extraction": {
    "enabled": true,
    "provider": "azure-content-understanding",
    "modelKey": "tasacion.1_0.azure-cu",
    "autoMapUnmappedFields": true,
    "fieldMappings": []
  },
  "fields": [ ... ]
}
```

Paso 4: Ajustar `fieldMappings` si es necesario

Si algún campo de la tipología tiene un nombre diferente al que devuelve Azure CU, añadirlo en `fieldMappings` (ver [sección 8](#)).

Paso 5: Verificar

No hay cambios de código. Ejecutar los tests existentes y arrancar la Function App en local apuntando al nuevo analizador.

6. Configuración en local (desarrollo)

Opción A: Usar el mock (sin llamada a Azure)

En `local.settings.json` asegurarse de que:

```
"Extraction:DefaultProvider": "mock"
```

Y en el fichero de validación de la tipología, poner `"provider": "mock"` (o dejarlo vacío para que use el default).

Opción B: Llamada real a Azure con API Key

1. Crear un recurso **Azure AI Services** en Azure Portal o Azure AI Foundry.
2. Obtener el **endpoint** (p. ej. `https://mi-recurso.cognitiveservices.azure.com/`) y una **API Key**.
3. Rellenar `local.settings.json` :

```
"Extraction:AzureContentUnderstanding:Endpoint": "https://mi-recurso.cognitiveservices.azure.com/",
"Extraction:AzureContentUnderstanding:ApiKey": "<api-key>",
"Extraction:AzureContentUnderstanding:AuthMode": "ApiKey"
```

4. Asegurarse de que el analizador referenciado en `models.json` (`analyzerId`) existe en ese recurso.

Opción C: Llamada real a Azure con identidad (DefaultAzureCredential)

1. Iniciar sesión con `az login` o Visual Studio.
2. Configurar:

```
"Extraction:AzureContentUnderstanding:Endpoint": "https://mi-recurso.cognitiveservices.azure.com/",
"Extraction:AzureContentUnderstanding:AuthMode": "DefaultAzureCredential"
```

3. La identidad local debe tener el rol **Cognitive Services User** en el recurso de Azure AI Services.

7. Configuración en Azure (producción)

Variables de entorno obligatorias en la Function App

Añadir en **Configuración > Variables de entorno** (o mediante Bicep/Terraform):

Variable	Valor
<code>Extraction__DefaultProvider</code>	<code>azure-content-understanding</code> (o <code>mock</code> para deshabilitar)
<code>Extraction__AzureContentUnderstanding__Endpoint</code>	Endpoint del recurso Azure AI Services
<code>Extraction__AzureContentUnderstanding__AuthMode</code>	<code>DefaultAzureCredential</code> (recomendado) o <code>ApiKey</code>
<code>Extraction__AzureContentUnderstanding__ApiKey</code>	Solo si <code>AuthMode=ApiKey</code> . Mejor usar un secreto de Key Vault.
<code>Extraction__AzureContentUnderstanding__DefaultProcessingLocation</code>	<code>global</code> (por defecto)

Nota: Azure Functions usa `__` como separador de jerarquía en variables de entorno (equivalente a `:` en JSON).

Autenticación recomendada en producción

Usar **Managed Identity** para evitar gestionar API keys:

1. Activar la **identidad administrada asignada por el sistema** en la Function App.
2. Asignar el rol **Cognitive Services User** a esa identidad en el recurso de Azure AI Services.
3. Configurar `AuthMode=DefaultAzureCredential` .

8. Mapeo de campos

Por defecto (`autoMapUnmappedFields: true`), el mapper intenta localizar cada campo declarado en `fields[]` de la tipología buscando en la respuesta de Azure un campo con el mismo nombre (sin distinción de mayúsculas).

Mapeo automático (por nombre)

Si el analizador devuelve un campo llamado `FincaRegistral` y la tipología tiene un campo llamado `FincaRegistral` , la asignación ocurre automáticamente sin configuración adicional.

Mapeo explícito con `fieldMappings`

Si el nombre del campo en el analizador de Azure difiere del nombre en la tipología, o si el valor está anidado dentro de un objeto o array, se usa `fieldMappings` :

```
"fieldMappings": [
  {
    "targetField": "Titular",
    "sourcePath": "Owner.Name"
  },
  {
    "targetField": "FechaInscripcion",
    "sourcePath": "InscriptionData.Date"
  }
]
```

- `targetField` : nombre del campo en la tipología (en `fields[]`).
- `sourcePath` : ruta en la respuesta de Azure usando notación de punto. Soporta:
 - Acceso directo: `CampoSimple`
 - Objetos anidados: `Objeto.SubCampo` (navega por `valueObject`)
 - Arrays indexados: `Lista.0.Nombre` (accede al primer elemento del `valueArray`)

Tipos de valor soportados

El mapper convierte automáticamente los tipos de Azure Content Understanding al tipo CLR correspondiente:

Tipo Azure CU	Tipo CLR	Ejemplo
<code>valueString</code>	<code>string</code>	<code>"Madrid"</code>
<code>valueDate</code>	<code>DateOnly</code>	<code>2024-03-15</code>
<code>valueDateTime</code>	<code>DateTimeOffset</code>	<code>2024-03-15T10:00:00Z</code>
<code>valueTime</code>	<code>TimeOnly</code>	<code>10:00:00</code>
<code>valuePhoneNumber</code>	<code>string</code>	<code>" +34 912 345 678"</code>
<code>valueNumber</code> / <code>valueInteger</code>	<code>double</code> / <code>long</code>	<code>123.45</code>
<code>valueBoolean</code>	<code>bool</code>	<code>true</code>
<code>valueArray</code>	<code>List<object></code>	<code>[...]</code>
<code>valueObject</code>	<code>Dictionary<string, object></code>	<code>{...}</code>
<code>content</code>	<code>string</code>	texto bruto del campo

9. Proveedor mock (sin Azure)

Para desarrollo local o entornos sin conexión a Azure, cualquier tipología puede usar el proveedor `mock` .

El `MockExtraerDataProvider` devuelve datos precargados en código (por tipología) sin hacer ninguna llamada externa. Es el comportamiento por defecto cuando `Extraction:DefaultProvider = mock` y la tipología no tiene `provider` explícito.

Para forzar el mock en una tipología específica aunque el default sea Azure:

```
"extraction": {
  "enabled": true,
  "provider": "mock",
  ...
}
```

10. Preguntas frecuentes

¿Qué pasa si el analizador de Azure no devuelve un campo que está declarado en la tipología? El campo simplemente no aparece en `DatosExtraidos`. No se produce ningún error. La validación posterior puede marcar ese campo como faltante si tiene `required: true`.

¿Qué pasa si `ModeloConfigs` no contiene la clave referenciada en la tipología? `ExtractionModelRegistryLoader.GetModel()` lanza `KeyNotFoundException` y la activity falla con un error descriptivo. El `models.json` físico solo serviría como seed de un entorno nuevo.

Cambios recientes (marzo 2026)

- Se ha actualizado el modelo registrado para la tipología `nota.simple.1_4` a un analizador real exportado: el `analyzerId` ahora es `CU_NS_1.4_2` y su `processingLocation` es `geography` (antes `nota-simple-1-4 / global`).
- Se confirmó mediante validación del schema del analizador (`CU_NS_1.4_2`) que los 28 campos principales coinciden con los nombres presentes en `nota.simple.1_4.validation.json`, por lo que el mapeo automático funciona sin `fieldMappings` adicionales.
- Ajustes de validación realizados en la tipología:
 - `CalificacionUrbanistica`: se añadió el valor `Urbana` al enum para cubrir un posible valor devuelto por el analizador.
 - `CuotaParticipacion`: se cambió la regex para aceptar formatos porcentuales producidos por el analizador (`100%`, `33.33%`, etc.). Regex actual: `^\\d+(\\.\\d+)?%$`.

Implicaciones:

- Actualiza `ModeloConfigs` con el `analyzerId` y `processingLocation` correctos antes de ejecutar en entorno apuntando al recurso Azure.
- Si se despliega en un entorno distinto (staging/prod), valida que el registro de BD de ese entorno contiene el modelo apropiado. Mantén `models.json` solo como seed o referencia.

¿Qué pasa si `Endpoint` no está configurado y se usa `azure-content-understanding`? `AzureContentUnderstandingProvider` lanza `InvalidOperationException` con el mensaje "Extraction:AzureContentUnderstanding:Endpoint es obligatorio" en el arranque de la Function App.

¿Se puede usar un analizador diferente para distintos entornos (dev/staging/prod)? Sí. El `analyzerId` está en `ModeloConfigs` para cada entorno. Se puede mantener un `models.json` diferente como seed inicial, pero la decisión operativa debe quedar publicada en BD.

¿La llamada a Azure es síncrona o asíncrona? La llamada usa `WaitUntil.Completed`, que hace que el SDK espere a que Azure finalice el análisis antes de devolver. Es una operación de larga duración (puede tardar segundos o minutos según el documento). Al estar dentro de una Durable Activity, el orquestador no se bloquea: la Activity se ejecuta en su propio worker y el orquestador continúa cuando recibe el resultado.

¿Cómo puedo ver el JSON raw que devuelve Azure? El log de la Function App con nivel `Debug` o `Trace` mostrará el JSON completo. También se puede instrumentar `AzureContentUnderstandingProvider` temporalmente para volcar `operation.Value.ToString()`.

11. Clasificación por defecto con Azure Document Intelligence

Desde marzo 2026, la clasificación del orquestador usa por defecto **Azure Document Intelligence** cuando no se indique lo contrario en el contrato de entrada.

Precedencia para clasificación:

- `Instrucciones.Classification.Provider` y `Instrucciones.Classification.Model` (si no son `auto`).

2. Classification:DefaultProvider y Classification:DefaultModelKey .

Regla importante de negocio:

- Si Instrucciones.ExpectedType viene informado, se **omite** la activity de clasificación y se fuerza:
 - Modelo = expectedtype-input
 - Confianza = 1.0
 - TipologiaDetectada = ExpectedType

Configuración mínima en appsettings.json / variables de entorno:

```
{
  "Classification": {
    "DefaultProvider": "azure-document-intelligence",
    "DefaultModelKey": "default.azure-di",
    "AzureDocumentIntelligence": {
      "Endpoint": "https://<tu-recurso>.cognitiveservices.azure.com/",
      "ApiKey": "<api-key>",
      "ApiVersion": "2024-11-30"
    }
  }
}
```

Registro de modelos de clasificación:

- Archivo: DocumentIA.Functions/config/classification/models.json
- Ejemplo:

```
{
  "models": [
    {
      "key": "default.azure-di",
      "provider": "azure-document-intelligence",
      "classifierId": "CHANGEME_CLASSIFIER_ID",
      "apiVersion": "2024-11-30"
    }
  ]
}
```

Nota de diseño:

- La tipología **no** selecciona el modelo de clasificación. La clasificación ocurre antes y es precisamente la que determina la tipología del documento.